# Qwyse AI Career Pods

---


## Team 6139

## Team Members

Gabrielle Oh, Arnav Ganga, David Martin and Jessica Ganji

## Client

Dr. Matthew Adeiza (CEO and Founder of Qwyse)


## Repository

https://github.com/phewee12/Team-6139---AI-Career-Pods

---

# Table of Contents


Qwyse AI Career Pods 1
- Client 1

Figures 3
Terminology 4
Background 5
Document Summary 6
Static System Architecture 7
- Introduction 7
- Rationale 8

Dynamic System Architecture 10
- Introduction 10
- Rationale 11

Data Design 12
- Introduction 12
- Entity Relationship Diagram 12
- File Use 13
- Data Exchange 13
- Data Security 13

UI Design 14
- Introduction 14
- Screen Descriptions 14
- Discover Groups 15
- Pod Feed 16
- Resume Review 17

Appendix 18

# Figures

Figure 1.1 Static System Architecture 9
Figure 1.2 Sequence Diagram 10
Figure 2.1 Entity Relationship Diagram 12
Figure 3.1 Discover Pods 15
Figure 3.2 Pod Feed 16
Figure 3.3 17

## Terminology

| Term | Definition | Context |
|---|---|---|
| AI Career Coach | The AI system that generates plans, summaries, insights, and motivational nudges for pod members. | Refers to the system's AI functionality. |
| Application Tracker | A shared system where users log job applications and track their status (Applied, Interview, Offer, etc.). | Used in Sprint 3 tracking features. |
| Cross-Pod Insights | Anonymized patterns or strategies extracted from pods. | Used in advanced intelligence features. |
| Mentor Matching | A feature that connects pods with alumni or mentors based on field alignment. | Used in community expansion features. |
| Node.js | A JavaScript runtime environment that executes server-side code outside of the browser. | Used to build backend services and business logic. |
| Peer Accountability Pod | A structured small group of 6–7 job-seeking students who collaborate weekly for accountability. | Core concept of the system. |
| PostgreSQL | An open-source relational database management system used to store structured data. | Stores pod data, user activity, goals, and engagement metrics. |
| React | A JavaScript library for building interactive and component-based user interfaces. | Used to build dashboards, trackers, and chat interfaces. |
| Weekly Summary | AI-generated recap of pod discussions, wins, and action items. | Generated at end of each accountability cycle. |

## Background

The job search today has become increasingly difficult leaving students and recent graduates in a competitive disarray. LinkedIn and other job boards provide access to apply for opportunities and network along with university career services; however, students and recent graduates are left with rejection fatigue, unsatisfactory results, lack of accountability, and uncertainty about job searching strategies. Research suggests that accountability, structured routines, a wide breadth of connections, and reinforcement techniques improve goal completion rates. However, this becomes an issue when most job boards treat job searching as an individual task rather than a collaborative process. Students are left confused and have a distaste towards job searching.

Qwyse' AI Career Pods aim to reinvent this space by introducing a collaborative space where students and recent graduates can have a structured and socially supported experience to perform their job searches. This platform will introduce pods that will contain 6-7 individuals and support features such as mock interviews, resume reviews, structured check-ins, and weekly goal setting. The integration of artificial intelligence will also further this model by deepening processes such as detecting disengagement and triggering motivational nudges and summarizing pod discussions into actionable insights. This combination of peer accountability and AI-driven behavioral support addresses a key gap effectively transforming this space into one where students and recent graduates look forward to engaging and thriving in.

# Document Summary

This Detailed Design Document outlines the technical and architectural design of the AI Career Pods system within Qwyse. The following sections describe how the system is structured, how components interact, how data is stored, and how users experience the platform.

**Static System Architecture**

This section describes the overall structural organization of our system. It goes into explaining the static elements of the backend and frontend systems. This section also goes through the introduction and rationale for our team's architectural choices and provides a high level view of our system and major components. Diagrams are included to show architectural components and explain logical relationships for our system.

**Dynamic System Architecture**

This section will describe the dynamic elements of the overall structural organization of our architecture. It will go through an introduction and rationale as well as include diagrams. **This will be included in Version 2.0 of our detailed design document.**

**Data Storage Design**

This section provides a clear understanding on how data is managed and exchanged within our systems. This section includes diagrams to provide clarity and go over the data protocol and format used for transferring data along with addressing any security concerns as well.

**UI Design**

This section will outline the user interface and design structures used when developing our application. We will include all major screens along with a heuristic analysis and design considerations. **This will be included in Version 2.0 of our detailed design document.**

# Static System Architecture

## Introduction

Our application is a progressive web application designed to integrate with the existing Qwyse platform. We are constructing our system using a layered architecture, allowing different teams to work on the frontend, backend, and AI components simultaneously without creating dependencies that block progress. Because this project will be embedded in the larger Qwyse ecosystem, we need a design that is adaptable so that when changes need to be made, we can update specific layers without having to deal with the entire codebase.

The following section shows a static view of our system architecture. Figure 1.0 illustrates the major components and their relationships, showing how data and control flow between layers. This diagram does not depict a specific user scenario but instead provides a high-level understanding of how the pieces fit together. Our system is organised into four main layers: the client layer containing our user-facing applications, the application services layer building core business logic, the AI services layer providing intelligent features, and the data layer managing persistence. The client layer includes a React-based PWA, a mobile-optimized version, and a browser extension. The application services layer is built with Node.js and contains services for authentication, user resume feedback, weekly summaries, and engagement analytics. The data layer relies on PostgreSQL accessed through Prisma ORM.

## Rationale

We selected a layered architecture to support future integration with Qwyse. This structure allows the frontend, backend, and database to work simultaneously without halting progression. Each layer has a well-defined responsibility, making the system easier to understand, test, and modify. If models need to be updated, those updates will remain isolated to a specific layer without affecting the entire codebase.

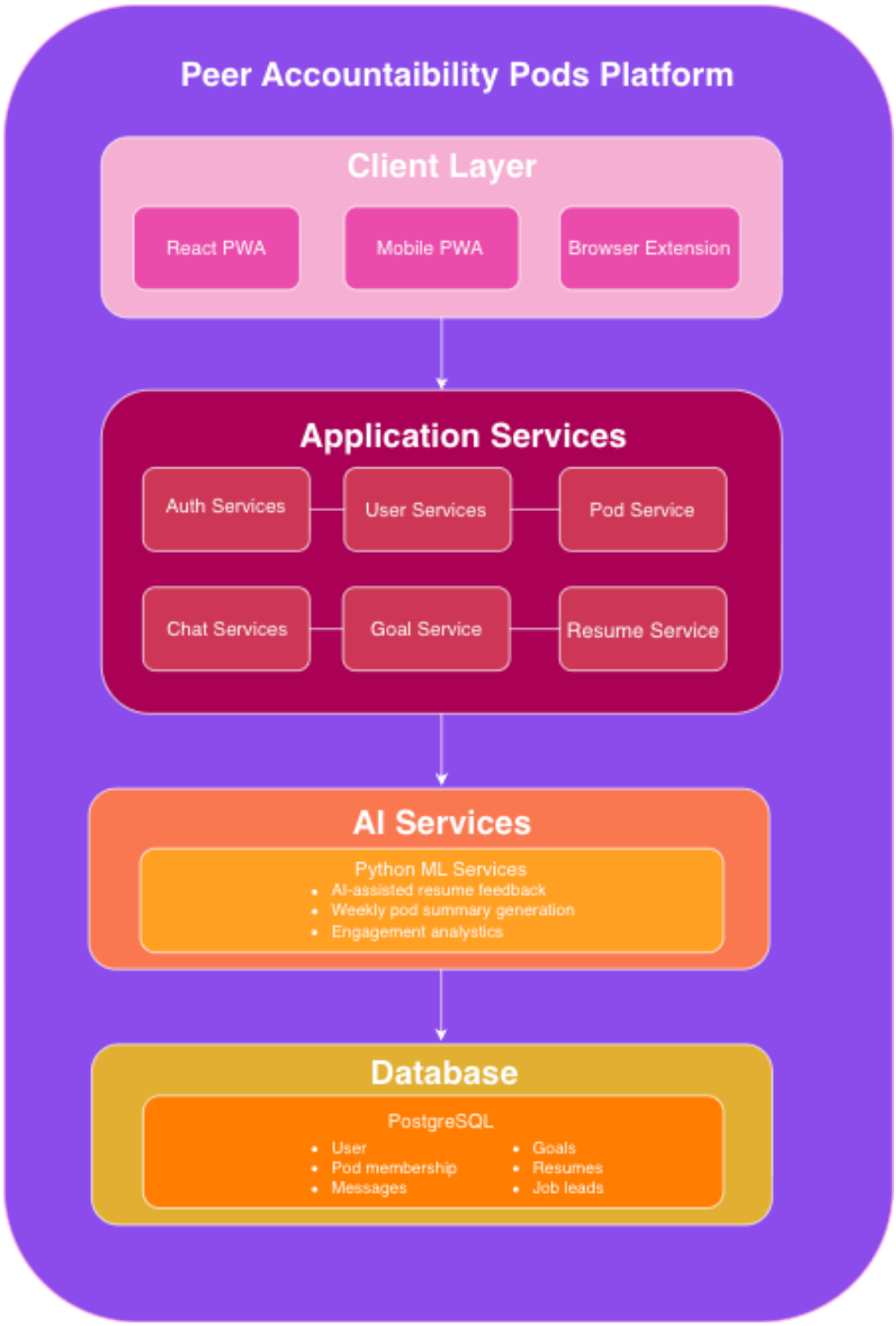
Our design choices were based on the compatibility with Qwyse's existing stack. The client layer uses React with TypeScript, matching Qwyse's frontend framework and enabling future component sharing. The application service layer is built with Node.js, mirroring Qwyse's backend infrastructure. All services use Prisma ORM for type-safe database access, ensuring consistency with Qwyse's data patterns. The AI services layer uses Python, leveraging its mature machine learning ecosystem for resume feedback, weekly summaries, and engagement analytics. The data later uses PostSQL, matching Qwyse's database choice and providing ACID compliance for data integrity with JSONB support for flexible schemas.

Our design is also influenced by security concerns. All sensitive data is stored in the database encrypted and files uploaded by users are scanned for viruses. We limit how many API requests to prevent abuse. Prisma protects against SQL injection by safely handling database queries. React automatically escapes content to prevent cross-site scripting attacks, and we use security headers for additional protection.

From the start we are ensuring privacy and protecting users' data. The system only asks for information we actually need. Chat messages are automatically removed after ninety days, and users can choose to opt out of AI features if they prefer.

The architecture also makes it easy to integrate with Qwyse later. We use the same frontend framework, backend technology, and database as Qwyse, so code can be shared or reused. Because the system is modular, Qwyse can adopt individual features over time without having to take everything at once.

# Diagrams



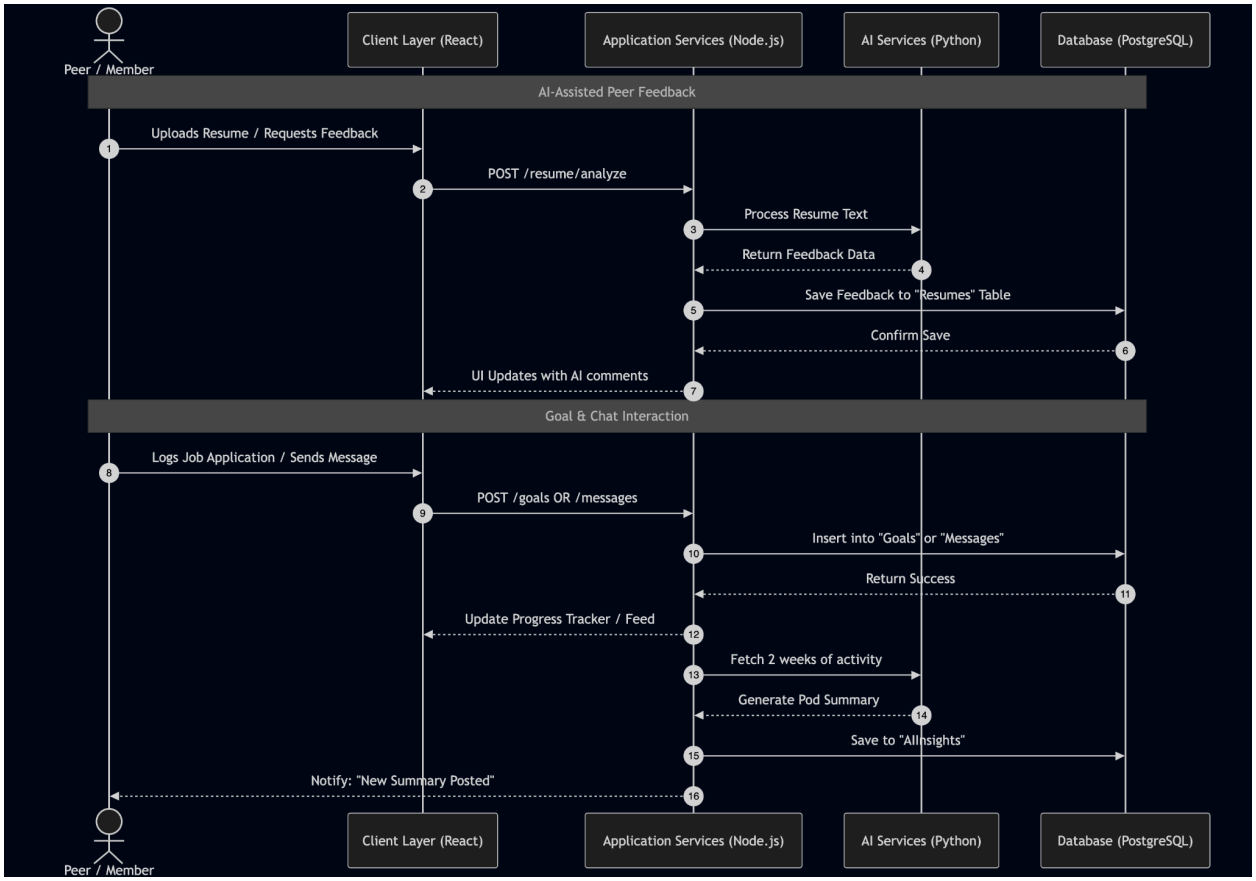

(Figure 1.1: Static System Diagram)

# Dynamic System Architecture

## Introduction

While the static architecture defines the structure and components of the AI Career Pods system, the dynamic architecture describes the behavioral patterns and communication flows that occur during runtime. This section illustrates how the layered components, specifically the React frontend, Node.js backend, and Python-based AI services, interact to fulfill specific user requirements.

By mapping the temporal sequence of messages and data exchanges, we can ensure that the system maintains high performance and data integrity during events, such as a pod-wide resume review or the automated generation of bi-weekly summaries.




(Figure 1.2: Sequence Diagram)

# Rationale

- **Instant Updates:** For simple things like sending a message or logging a job application, the React frontend talks straight to our Node.js server, which saves it to the PostgreSQL database. This happens almost instantly.
- **Handing off to AI:** For harder tasks like analyzing a resume, our Node.js server acts as a middleman. It sends the data to the Python AI service, waits for a response, and then saves it to the database for the user to see. This keeps the AI logic separate from our main app logic.
- **Scheduled Tasks:** We also have "set it and forget it" features. Our server is set up to automatically trigger the AI every two weeks to pull pod activity and generate those bi-weekly summaries.

## Runtime Behavior Breakdown

The sequence diagram above shows exactly how data moves through the app:

1. **User Actions:** When you upload a resume or post a goal, the **Client Layer (React)** sends that info to our **Application Services (Node.js)** using a POST request.
2. **Processing the Data:** If it's a resume, the Node.js server sends the text over to the **Python AI Services**. The AI doesn't talk to the database itself; it just sends the feedback back to our Node.js backend.
3. **Saving to the Database:** Once the backend has the final info (whether it’s a message or AI feedback), it uses Prisma to save everything into the **Postgres** tables.
4. **Closing the Loop:** After the database confirms the save, the server sends a "success" message back to the frontend. This is when the user sees their UI update with the new comment or goal progress.
5. **Background Summaries:** Every two weeks, the server automatically asks the AI to look at the last 14 days of data to create a pod summary. This happens in the background, and users just get a notification when it’s ready.

# Data Design

## Introduction

The AI Career Pods system uses PostgreSQL as its primary relational database and accesses it through Prisma ORM to ensure type-safe database interactions. PostgreSQL was selected due to its ACID compliance, mature ecosystem, support for relational integrity constraints, and JSONB functionality, which allows flexible storage of AI-generated metadata without requiring frequent schema redesign. The following subsections describe the database structure, file usage, data exchange mechanisms, and security considerations that govern how information flows through the system.

## Entity Relationship Diagram

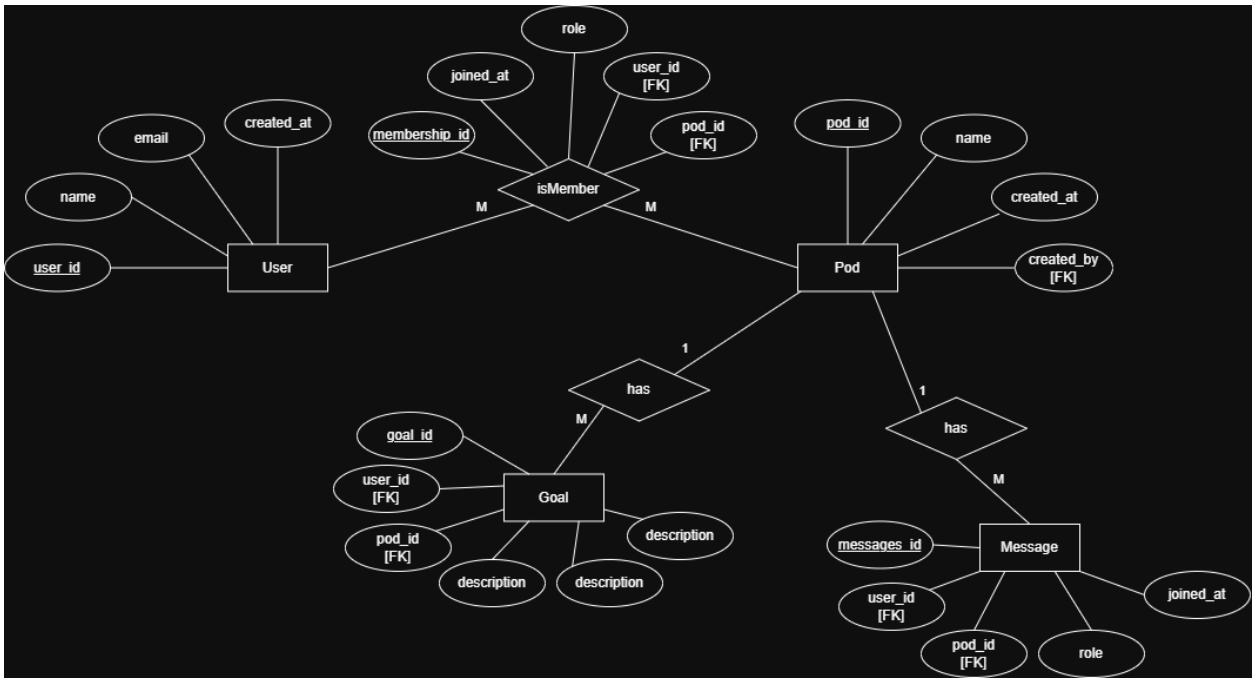



Figure 2.1: Entity Relationship Diagram

The core entities of the system include Users, Pods, Goals, Messages, and AIInsights. The Users table stores basic account information such as a unique user identifier, name, email address, password hash, account creation timestamp. Each user is uniquely identified using a UUID primary key. The Pods table stores information about each career pod, including a unique pod identifier, the pod name, creation timestamp, and the user who created it. Because a user can belong to multiple pods and a pod contains multiple users, a junction table named PodMembership resolves this many-to-many relationship. PodMembership contains foreign keys referencing both Users and Pods and records each member’s role and join date.

## File Use

Although the application is primarily database-driven, several file types are used within the system. Users may upload resumes in PDF or DOCX format to receive AI-driven feedback. These files are stored securely in object storage rather than directly in the relational database. Only metadata and storage references are saved in PostgreSQL.

Profile images are also supported and stored in secure object storage using common formats such as JPEG or PNG. Only the file path or URL is stored in the database. The system avoids storing large binary objects directly in PostgreSQL to preserve performance and scalability.

## Data Exchange

The AI Career Pods system uses a RESTful API architecture to facilitate communication between the client layer and the application services layer. All data exchanged between the frontend and backend is formatted using JSON (JavaScript Object Notation), which provides a lightweight, human-readable structure compatible with both JavaScript and Python services. Communication between the client and backend occurs over HTTPS, ensuring encrypted transmission through Transport Layer Security (TLS).

When AI services are invoked, the Node.js backend communicates with the Python-based AI services layer through secure internal API calls. These interactions also use JSON payloads. By standardizing on JSON and HTTPS, the system ensures consistent data serialization across all services while maintaining transport-level security.

## Data Security

Because the AI Career Pods platform processes personally identifiable information (PII) and behavioral engagement data, security is a core design priority. The system requires user authentication to access pods, submit goals, send messages, and receive AI-generated insights. Passwords are never stored in plain text. Session management is implemented using JSON Web Tokens (JWTs) stored in HTTP-only cookies to prevent cross-site scripting attacks.

All data transmitted between client devices and the backend server is encrypted using HTTPS. Data stored in PostgreSQL is protected through disk-level encryption, and files stored in object storage are encrypted at rest. Prisma ORM provides protection against SQL injection by parameterizing queries, and React’s automatic content escaping reduces cross-site scripting risks. Additional security headers are used to harden the application against common web vulnerabilities.

# UI Design

## Introduction

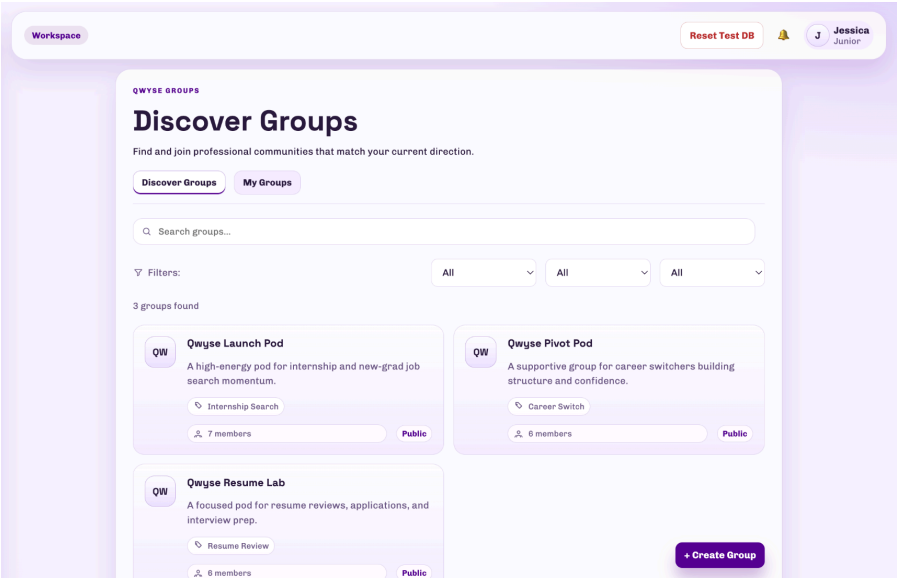
The user interface for **AI Career Pods** is designed to transform the often-isolating job search process into a collaborative, community-driven experience. Our design focuses on high-density information management that remains easy to navigate, ensuring students can track their goals, interact with peers, and access AI insights without feeling overwhelmed.

The following sections detail the core screens of the platform, including the group discovery phase and the internal pod workspace.

## Screen Descriptions & Heuristic Analysis

### 1. Discover Groups Screen

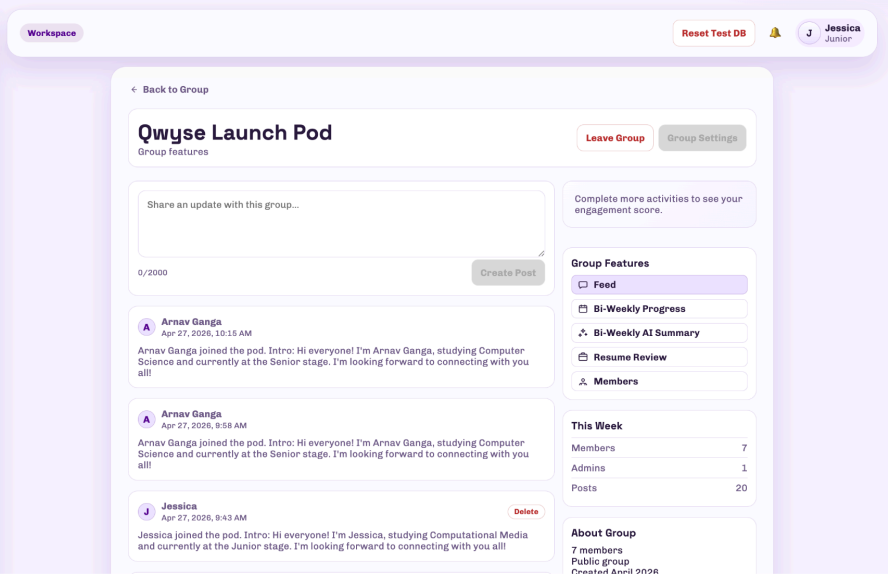
- **Description:** This is the entry point for users looking to find a community. It features a searchable directory of available "Pods" (e.g., Launch Pod, Pivot Pod, Resume Lab). Each card displays the pod's focus, member count, and tags like "Internship Search" to help users find the best fit.
- **Heuristic 1: Match between system and the real world.** The app uses familiar terms like "Pods," "Filters," and "Discover" that mirror how students look for clubs or study groups in real life. The "Public" badges and member counts make the digital experience feel like a physical career fair or community board.
- **Heuristic 2: Recognition rather than recall.** By using tags like "Internship Search" and "Career Switch" on the cards, users don't have to remember what each pod is for. The information is right there on the card, minimizing the mental effort needed to decide which group to join.
- **Heuristic 3: Aesthetic and minimalist design.** The screen isn't cluttered. It uses a clean card-based layout with plenty of white space, ensuring the user's focus stays on the group titles and descriptions rather than being distracted by unnecessary UI elements.




(Figure 3.1: Discover Pods)

**2. Pod Workspace (Feed)**

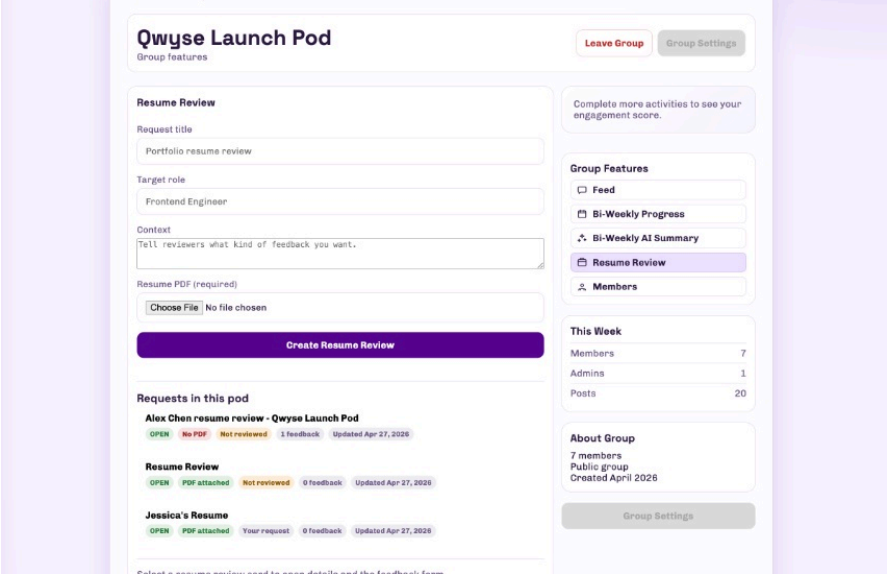
- **Description:** Once a user joins a pod, they land on this central dashboard. The left side manages navigation, while the main feed displays real-time updates, such as new members joining or peer posts. The right sidebar provides quick access to "Group Features" and high-level stats like post counts and member activity.
- **Heuristic 1: Consistency and standards.** The layout follows the "Sidebar + Main Feed + Right Sidebar" standard used by most professional tools (like Slack or Discord). Users already know that navigation is on the left and supplemental info is on the right, so there is no learning curve.
- **Heuristic 2: User control and freedom.** Each post has a "Delete" button clearly visible for the user's own content. This gives them a "way out" if they post something by mistake or want to remove an old intro message, providing a sense of control over their data.
- **Heuristic 3: Flexibility and efficiency of use.** The "Group Features" menu on the right acts as a set of shortcuts. While a user could find these features through the main nav, having them pinned right next to the feed allows frequent users to jump into the "Bi-Weekly AI Summary" or "Resume Review" with one click.




(Figure 3.2: Pod Feed)

### 3. Resume Review Interface

- **Description:** This screen allows users to request specific feedback by inputting a request title, target role, and context for reviewers. It also displays a list of active requests within the pod, showing status badges like "PDF attached" or "Reviewed" to keep the process transparent.
- **Heuristic 1: Visibility of system status.** The status badges (Open, No PDF, Not reviewed) are color-coded and highly visible. This tells the user exactly where their request stands in the pipeline without them having to click into each individual item to check.
- **Heuristic 2: Help users recognize, diagnose, and recover from errors.** The "Resume PDF (required)" label and the "No file chosen" text under the upload button act as a safeguard. If a user tries to create a review without a file, the UI makes it obvious where the missing piece is before they even hit submit.
- **Heuristic 3: Error prevention.** By including a "Target role" and "Context" box, the system prevents "bad" requests. It forces the user to provide enough information for the AI and peers to give helpful feedback, preventing the common error of getting vague or useless reviews.




(Figure 3.3: Resume Review)

# Appendix

## Gemini API 2.5 Flash

Rate Limits: 500 per day/10-15 per minute

## Team 6139

Gabrielle Oh (UI Lead)
goh33@gatech.edu

Jessica Ganji (Project Manager)
jganji3@gatech.edu

Arnav Ganga (Technical Developer)
aganga6@gatech.edu

David Martin (Architect)
dmartin305@gatech.edu